

PROJECT REPORT

Pearls AQI Predictor

An End-to-End Machine Learning System for 3-Day Air Quality Index Forecasting in Karachi, Pakistan

Prepared for: 10Pearls Shine Internship Program

Repository: github.com/SanaJamal-45/Pearls_AQI_Predictor

Dashboard Link: <https://pearlsaqipredictor.streamlit.app>

Date: September 2026

1. Executive Summary

The Pearls AQI Predictor is an end-to-end machine learning system that forecasts Karachi's Air Quality Index (AQI) three days into the future. The project covers the complete ML lifecycle: automated data collection, feature engineering, model training and evaluation, a feature store / model registry, CI/CD automation, and a user-facing web application with model explainability.

The system ingests live weather and pollutant data from the OpenWeather API, engineers a 28-feature set (lags, rolling averages, and change rates built on top of raw pollutant and weather readings), and trains four families of forecasting models (Ridge Regression, Random Forest, a PyTorch neural network, and tuned XGBoost) across three forecast horizons (t+1, t+2, t+3 days). XGBoost was selected as the production model after a documented, metric-driven comparison.

Two GitHub Actions workflows automate the pipeline end-to-end: an hourly feature pipeline and a daily retraining pipeline, both integrated with Hopsworks as the feature store and model registry. Predictions are served through a FastAPI backend (deployed on Render) and an interactive Streamlit dashboard that includes SHAP-based explainability and hazardous air quality alerts.

2. Project Objective

The goal of the project was to build a production-style, automated AQI forecasting system rather than a single one-off notebook model. Specifically, the system needed to:

- Continuously collect live pollutant and weather data for Karachi.
- Engineer meaningful predictive features from raw time-series readings.
- Train and rigorously compare multiple ML model families across three forecast horizons (next day, two days out, three days out).
- Persist features and trained models in a centralized feature store / model registry.
- Automate the entire pipeline on a schedule using CI/CD.
- Expose predictions through both an API (FastAPI) and an interactive dashboard (Streamlit), with explainability and alerting.

3. Tools & Technology Stack

Category	Tools / Technologies Used
Programming Language	Python
Machine Learning	scikit-learn, XGBoost, PyTorch
Feature Store / Model Registry	Hopsworks
CI / CD & Automation	GitHub Actions (scheduled workflows)

Category	Tools / Technologies Used
Backend / API	FastAPI, Uvicorn
Dashboard / Visualization	Streamlit, Plotly
Model Explainability	SHAP (SHapley Additive exPlanations)
Data Sources	OpenWeather API (live pollution + weather); historical Karachi daily AQI/weather dataset
Data Handling	pandas, NumPy
Model Persistence	joblib (Ridge, XGBoost), PyTorch .pth checkpoints
Version Control	Git / GitHub
Deployment	Streamlit Cloud (dashboard), Render (FastAPI backend)

4. System Architecture Overview

The project follows a modular, serverless-pipeline architecture composed of six main stages:

- Data Collection : live pollutant and weather data pulled from the OpenWeather API on an hourly schedule.
- Feature Engineering : raw readings converted into a 28-column model-ready feature set.
- Feature Store : engineered features written to a Hopsworks feature group for versioned, centralized storage.
- Model Training & Registry : multiple models trained per forecast horizon, with the best performer (XGBoost) registered and versioned in the Hopsworks Model Registry.
- Automation : two GitHub Actions workflows keep data and models continuously up to date.
- Serving Layer : a FastAPI backend (deployed on Render) and a Streamlit dashboard expose predictions, explanations, and alerts to end users.

5. Data Collection & Sources

5.1 Historical Backfill

The historical foundation of the project is a Karachi daily AQI and weather dataset spanning roughly 3.5 years (over 1,300 daily rows), stored at `data/raw/karachi_daily_aqi_weather.csv`. Each row contains the day's AQI along with the underlying pollutant concentrations (PM2.5, PM10, NO2, SO2, CO, O3) and weather readings (temperature, humidity, precipitation).

5.2 Live Data Feed

Ongoing live data is collected from the OpenWeather API via `src/openweather_collector.py`, which fetches forecasted pollutant concentrations (PM2.5, PM10, NO2, SO2, CO, O3) and weather variables (temperature, humidity, precipitation). Because OpenWeather's own air-quality index uses a different 1–5

scale than the EPA-style AQI used in the historical dataset, `src/openweather_features.py` re-derives AQI from PM2.5/PM10 concentrations using the standard EPA breakpoint formula, rather than relying on OpenWeather's built-in index, this keeps the live feed consistent with the historical training data.

6. Feature Engineering

Feature engineering (`src/feature_engineering.py`) transforms the raw daily readings into a rich, date-aware feature set. Because some calendar dates are missing from the underlying series, all rolling and lag calculations are computed using actual calendar dates rather than row position, so a data gap is never silently treated as consecutive days.

6.1 Feature Categories

- Raw readings: AQI, PM2.5, PM10, NO2, SO2, CO, O3, Temperature, Humidity, Precipitation.
- Calendar / time-based: `day`, `day_of_week`, `month`, `year`, `day_of_year`.
- Lag features: `AQI_lag_1`, `AQI_lag_2`, `AQI_lag_3`, `AQI_lag_7` (AQI value 1, 2, 3, and 7 days prior).
- Change features: `AQI_change_1d`, `AQI_change_3d`, `AQI_change_7d` (difference vs. the corresponding lag).
- Rolling averages: `AQI_rolling_mean_3/7`, `PM2.5_rolling_mean_3/7`, `PM10_rolling_mean_3/7`, computed over the trailing 3 and 7 calendar days (excluding the current day).

6.2 Prediction Targets

Three forward-looking targets are generated by looking up the AQI value on future calendar dates: `AQI_t+1`, `AQI_t+2`, and `AQI_t+3`, representing the AQI one, two, and three days ahead respectively. This turns the problem into three parallel supervised regression tasks — one model is trained per horizon.

After feature engineering, the processed dataset (`data/processed/aqi_features.csv`) contains 1,319 rows and 32 columns, ready for model training.

7. Train / Validation / Test Split

The dataset was split strictly chronologically (`src/split_data.py`) to avoid lookahead / data leakage which is a critical requirement for any time-series forecasting problem:

Split	Date Range	Purpose
Train	Before 2026-01-01	Model fitting
Validation	2026-01-01 to 2026-06-30	Hyperparameter tuning & model selection
Test	2026-07-01 onward	Final, held-out performance evaluation

Rows with missing required features (e.g., early rows without enough history for a 7-day lag) were dropped before splitting, ensuring every training example is fully populated.

8. Model Development & Training

Four distinct model families were trained and evaluated independently for each of the three forecast horizons (t+1, t+2, t+3), giving a broad comparison from simple linear models to deep learning:

8.1 Ridge Regression (Baseline / Linear)

A regularized linear regression model (`src/ridge_model.py`) served as an interpretable baseline. It performed strongly at the 1-day horizon but degraded sharply at longer horizons.

8.2 Random Forest

An ensemble of decision trees (`src/randomforest.py`) was trained to capture non-linear relationships between pollutants, weather, and future AQI, offering a middle ground between the linear baseline and gradient boosting.

8.3 PyTorch Neural Network

A neural network built with PyTorch (`src/pytorch.py`) was trained per horizon. With only roughly 175 rows in the validation set, the neural network was expected to underperform simpler, better-regularized models — a hypothesis borne out in the final results.

8.4 XGBoost (Tuned) — Production Model

A gradient-boosted tree model (`src/xgboost_tuned.py`) was tuned separately for each forecast horizon using `RandomizedSearchCV` with 60 iterations and 3-fold cross-validation. The resulting best hyperparameters differed meaningfully by horizon — the 1-day model favored more trees at a low learning rate with no subsampling, while the 2- and 3-day models used heavier regularization and row/column subsampling to fight overfitting on the harder, noisier longer-horizon targets.

Horizon	n_estimators	max_depth	learning_rate	subsample	colsample_bytree	reg_alpha	reg_lambda
t+1	500	2	0.05	1.0	1.0	0.5	5.0
t+2	300	2	0.01	0.6	0.7	1.0	2.0
t+3	300	2	0.01	0.6	0.7	1.0	2.0

Table: Best XGBoost hyperparameters per forecast horizon, selected via `RandomizedSearchCV`.

9. Model Evaluation & Results

All models were evaluated with three standard regression metrics — Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and R^2 (coefficient of determination) — on the held-out validation and test splits.

9.1 R^2 Comparison Across Models and Horizons

Model	t+1 R ²	t+2 R ²	t+3 R ²
Ridge Regression	0.844	0.32	-0.09
Random Forest	0.79	0.36	0.26
PyTorch Neural Network	0.38 – 0.57	~0.16	~-0.05
XGBoost (Tuned)	0.851	0.363 – 0.41	0.106 – 0.18

Table: R² scores by model and forecast horizon (higher is better; XGBoost values in bold in the source project).

9.2 Key Findings

- XGBoost (tuned) was selected as the production model — it matches or slightly beats Ridge at the 1-day horizon, and meaningfully improves the weaker 2-day and 3-day horizons, which were the main accuracy problem in earlier iterations.
- Ridge's 3-day R² was negative (-0.09), meaning it performed worse than simply guessing the historical average for that horizon — a clear sign that a purely linear model cannot capture the compounding uncertainty of a 3-day-ahead forecast.
- PyTorch underperformed across the board. This is attributed to the small validation set (~175 rows), which is too small for a neural network to reliably outperform a well-regularized model such as XGBoost or Ridge.
- Predictive accuracy naturally decreases with forecast distance for every model — 1-day-ahead predictions are the most reliable, and 3-day-ahead predictions carry the most uncertainty, which is expected for weather/pollution-driven forecasting.

9.3 Feature Importance

Feature importance analysis (via XGBoost's built-in importances and SHAP) confirmed that PM2.5 is the dominant driver of predicted AQI, accounting for roughly 36% of total feature importance, consistent with atmospheric science, since PM2.5 is well known to be the primary pollutant driving urban air quality in cities like Karachi.

10. Feature Store & Model Registry (Hopsworks)

- A feature group named `aqi_features` (version 1) stores the full historical backfill (1,319 rows, 32 columns) plus live daily inserts from the automated pipeline.
- A Model Registry holds the production XGBoost models — `aqi_xgboost_t_plus_1`, `aqi_xgboost_t_plus_2`, `aqi_xgboost_t_plus_3` — each versioned with its RMSE, MAE, and R² metrics attached.

11. Automation & CI/CD (GitHub Actions)

Two scheduled GitHub Actions workflows fully automate the pipeline, and both can also be triggered manually via `workflow_dispatch`:

Workflow	Schedule	What It Does
feature_pipeline.yml	Hourly	Fetches live OpenWeather data → builds model-ready features → inserts them into the Hopsworks feature store. Verified working end-to-end.
training_pipeline.yml	Daily	Retrains XGBoost on the latest available data → evaluates it → pushes new model versions to the Hopsworks Model Registry. Verified working end-to-end.

12. Web Application

Two independent serving components were built, deliberately satisfying both a Streamlit/Gradio-style interface and a Flask/FastAPI-style API requirement:

12.1 FastAPI Backend (app/backend.py)

The FastAPI backend is deployed on **Render** and provides the following endpoints:

- GET /health — service health check.
- GET /history?days=N — historical AQI records.
- GET /predict/latest and POST /predict — returns the 3-day forecast; POST supports "what-if" feature overrides.
- GET /explain/latest and POST /explain — local SHAP explanation for a single prediction.
- GET /explain/global — global SHAP feature-importance summary across a sample of historical rows.

Run locally with: `uvicorn app.backend:app --reload --port 8000`

12.2 Streamlit Dashboard (app/dashboard.py)

- The dashboard is deployed on Streamlit Cloud at <https://pearlsaqipredictor.streamlit.app>. In the deployed environment, the dashboard connects directly to the FastAPI backend (running on Render) to fetch historical data, generate 3-day forecasts, and retrieve SHAP explanations via REST API calls. This ensures that the live predictions and model logic are always served by the production-ready backend.
- For local development, the dashboard also includes a standalone fallback mode that imports prediction and SHAP logic directly from `app/model_utils.py`. This allows it to run independently without a live backend connection, making local testing and debugging convenient. Historical AQI trend chart with an adjustable date range.
- Its features include:
 - 3-day forecast cards, color-coded by EPA AQI category.
 - A hazardous air quality alert banner that triggers when any horizon crosses into "Unhealthy" or worse.

- "What-if" sliders (temperature, PM2.5, humidity) to explore hypothetical scenarios interactively.
- SHAP explainability, offering both a per-prediction local explanation and an overall global feature-importance view.

Run locally with: `streamlit run app/dashboard.py`

13. Advanced Analytics

- Exploratory Data Analysis: distribution, seasonal, day-of-week, correlation, and lag-relationship analysis performed on the historical dataset.
- SHAP Explainability : implemented via `shap.TreeExplainer`, providing both local (per-forecast) and global (overall) feature-importance views, integrated directly into the dashboard.
- Hazardous AQI Alerts : the dashboard automatically raises a warning/danger banner whenever any of the three forecast horizons crosses into "Unhealthy" territory or worse.
- Multi-model comparison : from linear (Ridge) through tree ensembles (Random Forest, XGBoost) to deep learning (PyTorch), with a documented, data-driven model selection rather than defaulting to one algorithm.

14. Health Advisory Feature

A critical component of the dashboard is the **Health Advisory System**, which uses the EPA AQI breakpoints to classify forecasted air quality and issue actionable alerts:

- **Alert Levels:**
 - **Green (Good)** — no alert.
 - **Yellow (Moderate)** — caution for sensitive groups.
 - **Orange (Unhealthy for Sensitive Groups)** — advisory for children, elderly, and respiratory patients.
 - **Red (Unhealthy)** — warning banner: "Air quality is unhealthy – limit outdoor activities."
 - **Purple (Very Unhealthy)** — urgent alert: "Avoid all outdoor exertion."
 - **Maroon (Hazardous)** — emergency alert: "Stay indoors, use air purifiers."
- **Implementation:** The dashboard calculates the AQI category for each forecast horizon and displays a persistent banner if any day falls into "Unhealthy" or worse. The banner includes a recommendation based on the worst category.
- **Health-Based Recommendations:** For each alert level, the system provides simple, actionable guidance (e.g., "Wear a mask if you must go outside," "Keep windows closed"), making the tool useful for public health communication.

15. Challenges Encountered

- A 3-day data gap between the historical dataset and the live OpenWeather feed, resulting in partial lag features for a small number of rows.
- Persistent, reproducible Hopsworks Arrow Flight read failures, requiring a local-CSV fallback for the retraining pipeline while keeping writes to the feature store and model registry intact.
- Exhaustion of the Hopsworks free-tier compute budget mid-project, requiring the dashboard and API to be designed for resilience by serving from local files rather than depending solely on the live feature store.
- A small validation set (~175 rows) that limited how well a deep learning model (PyTorch) could compete with simpler, regularized models.
- Declining predictive power at longer forecast horizons (t+2, t+3), an inherent challenge of multi-day air quality forecasting that was addressed through per-horizon hyperparameter tuning rather than a one-size-fits-all model.

16. Conclusion

The Pearls AQI Predictor successfully delivers a complete, production-style machine learning system for forecasting Karachi's air quality up to three days in advance. Beyond model training, the project demonstrates the full MLOps lifecycle: automated hourly data collection, robust date-aware feature engineering, rigorous chronological train/validation/test evaluation across four model families, a feature store and model registry integration, scheduled CI/CD retraining, and a dual-interface serving layer (API + dashboard) with built-in explainability and safety alerting.

Through a transparent, metrics-driven comparison, XGBoost was selected as the production model for its consistently stronger performance across all three forecast horizons, and its feature-importance results (PM2.5 as the dominant driver) align well with known atmospheric science for pollution-driven cities.

Overall, the project reflects strong end-to-end data science and ML engineering practice: sound data handling, disciplined evaluation methodology, automated infrastructure, and a usable, explainable end-user product.

17. Future Improvements

- Explore additional exogenous features (e.g., traffic density, industrial activity, wind direction) to improve 2- and 3-day forecast accuracy.
- Add automated model-performance monitoring/alerting to detect drift after each daily retraining run.